

Exercise 1: Control Structures

Scenario 1: The bank wants to apply a discount to loan interest rates for customers above 60 years old.

- **Question:** Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

```
SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
2  BEGIN
3      FOR rec IN (
4          SELECT c.CustomerID
5              FROM Customers c
6              WHERE TRUNC(MONTHS_BETWEEN(SYSDATE, c.DOB)/12) > 60
7      )
8      LOOP
9          UPDATE Loans
10             SET InterestRate = InterestRate - 1
11             WHERE CustomerID = rec.CustomerID;
12      END LOOP;
13
14      COMMIT;
15      DBMS_OUTPUT.PUT_LINE('Interest rates updated successfully.');
```

16 END;

17 /

Interest rates updated successfully.

PL/SQL procedure successfully completed.

Scenario 2: A customer can be promoted to VIP status based on their balance.

- **Question:** Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over \$10,000.

```

SQL> set serveroutput on;
SQL> DECLARE
2 BEGIN
3     FOR rec IN (
4         SELECT CustomerID
5         FROM Customers
6         WHERE Balance > 10000
7     )
8     LOOP
9         UPDATE Customers
10            SET IsVIP = 'TRUE'
11            WHERE CustomerID = rec.CustomerID;
12    END LOOP;
13
14    COMMIT;
15    DBMS_OUTPUT.PUT_LINE('VIP status updated successfully.');
```

VIP status updated successfully.

PL/SQL procedure successfully completed.

○

Scenario 3: The bank wants to send reminders to customers whose loans are due within the next 30 days.

- **Question:** Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

```

SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
2 BEGIN
3     FOR rec IN (
4         SELECT c.CustomerID, l.LoanID, l.EndDate
5         FROM Customers c
6         JOIN Loans l
7         ON c.CustomerID = l.CustomerID
8         WHERE l.EndDate BETWEEN SYSDATE AND SYSDATE + 30
9     )
10    LOOP
11        DBMS_OUTPUT.PUT_LINE(
12            'Reminder: Customer ' || rec.CustomerID ||
13            ', Loan ' || rec.LoanID ||
14            ' is due on ' || TO_CHAR(rec.EndDate, 'DD-MON-YYYY')
15        );
16    END LOOP;
17 END;
18 /
```

PL/SQL procedure successfully completed.

○

Exercise 2: Error Handling

Scenario 1: Handle exceptions during fund transfers between accounts.

- **Question:** Write a stored procedure **SafeTransferFunds** that transfers funds between two accounts. Ensure that if any error occurs (e.g., insufficient funds), an appropriate error message is logged and the transaction is rolled back.

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> CREATE OR REPLACE PROCEDURE SafeTransferFunds(
  2     p_fromAccount IN Accounts.AccountID%TYPE,
  3     p_toAccount   IN Accounts.AccountID%TYPE,
  4     p_amount      IN Accounts.Balance%TYPE
  5 )
  6 IS
  7     v_balance Accounts.Balance%TYPE;
  8 BEGIN
  9     SELECT Balance
 10     INTO v_balance
 11     FROM Accounts
 12     WHERE AccountID = p_fromAccount;
 13     IF v_balance < p_amount THEN
 14         RAISE_APPLICATION_ERROR(-20001,
 15         'Insufficient funds in source account.');
```

```
16     END IF;
17     UPDATE Accounts
18     SET Balance = Balance - p_amount
19     WHERE AccountID = p_fromAccount;
20     UPDATE Accounts
21     SET Balance = Balance + p_amount
22     WHERE AccountID = p_toAccount;
23
24     COMMIT;
25     DBMS_OUTPUT.PUT_LINE('Fund transfer successful.');
```

```
26
27 EXCEPTION
28     WHEN OTHERS THEN
29         ROLLBACK;
30         DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
31 END;
32 /
```

○

Scenario 2: Manage errors when updating employee salaries.

- **Question:** Write a stored procedure **UpdateSalary** that increases the salary of an employee by a given percentage. If the employee ID does not exist, handle the exception and log an error message.

```

SQL> SET SERVEROUTPUT ON;
SQL>
SQL> CREATE OR REPLACE PROCEDURE UpdateSalary(
  2     p_empid      IN Employees.EmployeeID%TYPE,
  3     p_percentage IN NUMBER
  4 )
  5 IS
  6 BEGIN
  7     UPDATE Employees
  8     SET Salary = Salary + Salary * p_percentage / 100
  9     WHERE EmployeeID = p_empid;
 10
 11     IF SQL%ROWCOUNT = 0 THEN
 12         RAISE NO_DATA_FOUND;
 13     END IF;
 14
 15     COMMIT;
 16
 17     DBMS_OUTPUT.PUT_LINE('Salary updated successfully.');
```

```

 18
 19 EXCEPTION
 20     WHEN NO_DATA_FOUND THEN
 21         DBMS_OUTPUT.PUT_LINE(
 22             'Error: Employee ID ' || p_empid || ' does not exist.'
 23         );
 24         ROLLBACK;
 25
 26     WHEN OTHERS THEN
 27         DBMS_OUTPUT.PUT_LINE(
 28             'Error: ' || SQLERRM
 29         );
 30         ROLLBACK;
 31 END;
 32 /

```

Procedure created.

```

SQL> SET SERVEROUTPUT ON;
SQL> BEGIN
  2     UpdateSalary(1,10);
  3 END;
  4 /
Salary updated successfully.

PL/SQL procedure successfully completed.

SQL> select * from employees;

```

```

EMPLOYEEID
-----
NAME
-----
POSITION                                SALARY
-----
DEPARTMENT                                HIREDATE
-----

```

1	Alice Johnson	77000
	Manager	
	HR	15-JUN-15

Scenario 3: Ensure data integrity when adding a new customer.

- **Question:** Write a stored procedure **AddNewCustomer** that inserts a new customer into the Customers table. If a customer with the same ID already exists, handle the exception by logging an error and preventing the insertion.

```
SQL> SET SERVEROUTPUT ON;
SQL> CREATE OR REPLACE PROCEDURE ADDNEWCUSTOMER(
  2   P_Customerid IN Customers.CustomerID%TYPE,
  3   P_Name IN customers.Name%TYPE,
  4   p_Dob IN Customers.dob%TYPE,
  5   P_Balance IN Customers.balance%TYPE) IS
  6   begin
  7     INSERT INTO Customers(CustomerID, Name, DOB, Balance)
  8       VALUES (p_customerid, p_name, p_dob, p_balance);
  9
 10     COMMIT;
 11
 12     DBMS_OUTPUT.PUT_LINE('Customer added successfully.');
```

```
13 EXCEPTION
14   WHEN DUP_VAL_ON_INDEX THEN
15     DBMS_OUTPUT.PUT_LINE(
16       'Error: Customer ID ' || p_customerid || ' already exists.'
17     );
18     ROLLBACK;
19
20   WHEN OTHERS THEN
21     DBMS_OUTPUT.PUT_LINE(
22       'Error: ' || SQLERRM
23     );
24     ROLLBACK;
25 END;
26 /
```

Procedure created.

○

```

SQL> SET SERVEROUTPUT ON;
SQL>
SQL> BEGIN
2      AddNewCustomer(
3          3,
4          'John',
5          TO_DATE('15-05-2000', 'DD-MM-YYYY'),
6          5000
7      );
8  END;
9  /
Customer added successfully.

PL/SQL procedure successfully completed.

SQL> select * from customers;

CUSTOMERID
-----
NAME
-----
DOB          BALANCE  LASTMODIF  ISVIP
-----
1
John Doe
15-MAY-85      1000  23-JUN-26
2
Jane Smith
20-JUL-90      1500  23-JUN-26
CUSTOMERID
-----
NAME
-----
DOB          BALANCE  LASTMODIF  ISVIP
-----
3
John
15-MAY-00      5000

```

○

Exercise 3: Stored Procedures

Scenario 1: The bank needs to process monthly interest for all savings accounts.

- **Question:** Write a stored procedure **ProcessMonthlyInterest** that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

```

SQL> SET SERVEROUTPUT ON;
SQL>
SQL> CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
2  IS
3  BEGIN
4      UPDATE Accounts
5      SET Balance = Balance * 1.01
6      WHERE AccountType = 'SAVINGS';
7
8      COMMIT;
9
10     DBMS_OUTPUT.PUT_LINE(
11         'Monthly interest applied successfully.'
12     );
13
14 EXCEPTION
15     WHEN OTHERS THEN
16         ROLLBACK;
17
18         DBMS_OUTPUT.PUT_LINE(
19             'Error: ' || SQLERRM
20         );
21 END;
22 /

```

Procedure created.

```
SQL> SET SERVEROUTPUT ON;
```

```
SQL>
```

```
SQL> BEGIN
```

```
2     ProcessMonthlyInterest;
```

```
3 END;
```

```
4 /
```

Monthly interest applied successfully.

PL/SQL procedure successfully completed.

```

SQL> SELECT AccountID,
2         CustomerID,
3         AccountType,
4         Balance
5 FROM Accounts;

```

ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE
1	1	Savings	1000
2	2	Checking	1500

Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance.

- **Question:** Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

```

SQL> SET SERVEROUTPUT ON;
SQL>
SQL> CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
  2     p_department IN Employees.Department%TYPE,
  3     p_bonus_pct  IN NUMBER
  4 )
  5 IS
  6 BEGIN
  7     UPDATE Employees
  8     SET Salary = Salary + Salary * p_bonus_pct / 100
  9     WHERE Department = p_department;
 10
 11     COMMIT;
 12
 13     DBMS_OUTPUT.PUT_LINE(
 14         'Bonus updated successfully.'
 15     );
 16
 17 EXCEPTION
 18     WHEN OTHERS THEN
 19         ROLLBACK;
 20
 21         DBMS_OUTPUT.PUT_LINE(
 22             'Error: ' || SQLERRM
 23         );
 24 END;
 25 /

```

Procedure created.

○

```

SQL> SET SERVEROUTPUT ON;
SQL>
SQL> BEGIN
  2     UpdateEmployeeBonus('IT',10);
  3 END;
  4 /
Bonus updated successfully.

PL/SQL procedure successfully completed.

SQL> select * from employees;

EMPLOYEEID
-----
NAME
-----
POSITION                                SALARY
-----
DEPARTMENT                             HIREDATE
-----
1
Alice Johnson
Manager
HR
77000
15-JUN-15

EMPLOYEEID
-----
NAME
-----
POSITION                                SALARY
-----
DEPARTMENT                             HIREDATE
-----
2
Bob Brown
Developer
IT
66000
20-MAR-17

```

○

Scenario 3: Customers should be able to transfer funds between their accounts.

- **Question:** Write a stored procedure **TransferFunds** that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.
-

```

SQL> SET SERVEROUTPUT ON;
SQL>
SQL> CREATE OR REPLACE PROCEDURE TransferFunds(
  2     p_from_account IN Accounts.AccountID%TYPE,
  3     p_to_account   IN Accounts.AccountID%TYPE,
  4     p_amount       IN Accounts.Balance%TYPE
  5 )
  6 IS
  7     v_balance Accounts.Balance%TYPE;
  8 BEGIN
  9     SELECT Balance
 10     INTO v_balance
 11     FROM Accounts
 12     WHERE AccountID = p_from_account;
 13
 14     IF v_balance < p_amount THEN
 15         RAISE_APPLICATION_ERROR(
 16             -20001,
 17             'Insufficient balance.'
 18         );
 19     END IF;
 20
 21     UPDATE Accounts
 22     SET Balance = Balance - p_amount,
 23         LastModified = SYSDATE
 24     WHERE AccountID = p_from_account;
 25
 26     UPDATE Accounts
 27     SET Balance = Balance + p_amount,
 28         LastModified = SYSDATE
 29     WHERE AccountID = p_to_account;
 30
 31     COMMIT;
 32
 33     DBMS_OUTPUT.PUT_LINE(
 34         'Fund transfer successful.'
 35     );
 36
 37 EXCEPTION
 38     WHEN OTHERS THEN
 39         ROLLBACK;
 40
 41         DBMS_OUTPUT.PUT_LINE(
 42             'Error: ' || SQLERRM
 43         );
 44 END;
 45 /

```

Procedure created.

```

SQL> SET SERVEROUTPUT ON;
SQL>
SQL> BEGIN
  2     TransferFunds(1,2,500);
  3 END;
  4 /
Fund transfer successful.

PL/SQL procedure successfully completed.

SQL> |

```

Exercise 4: Functions

Scenario 1: Calculate the age of customers for eligibility checks.

- **Question:** Write a function CalculateAge that takes a customer's date of birth as input and returns their age in years.

```
SQL> CREATE OR REPLACE FUNCTION CalculateAge(  
2     p_dob IN Customers.DOB%TYPE  
3 )  
4 RETURN NUMBER  
5 IS  
6     v_age NUMBER;  
7 BEGIN  
8     v_age := TRUNC(MONTHS_BETWEEN(SYSDATE, p_dob) / 12);  
9  
10    RETURN v_age;  
11 END;  
12 /  
  
Function created.  
  
SQL> SET SERVEROUTPUT ON;  
SQL>  
SQL> BEGIN  
2     DBMS_OUTPUT.PUT_LINE(  
3         'Age = ' ||  
4         CalculateAge(TO_DATE('15-05-2000', 'DD-MM-YYYY'))  
5     );  
6 END;  
7 /  
Age = 26  
  
PL/SQL procedure successfully completed.
```

○

Scenario 2: The bank needs to compute the monthly installment for a loan.

- **Question:** Write a function **CalculateMonthlyInstallment** that takes the loan amount, interest rate, and loan duration in years as input and returns the monthly installment amount.

```

SQL> CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment(
2     p_loan_amount    IN Loans.LoanAmount%TYPE,
3     p_interest_rate  IN Loans.InterestRate%TYPE,
4     p_years           IN NUMBER
5 )
6 RETURN NUMBER
7 IS
8     v_total_amount NUMBER;
9     v_emi NUMBER;
10 BEGIN
11     v_total_amount :=
12         p_loan_amount +
13         (p_loan_amount * p_interest_rate * p_years / 100);
14
15     v_emi := v_total_amount / (p_years * 12);
16
17     RETURN ROUND(v_emi,2);
18 END;
19 /

```

Function created.

```

SQL> SET SERVEROUTPUT ON;
SQL>
SQL> BEGIN
2     DBMS_OUTPUT.PUT_LINE(
3         'Monthly Installment = ' ||
4         CalculateMonthlyInstallment(100000,10,5)
5     );
6 END;
7 /

```

Monthly Installment = 2500

PL/SQL procedure successfully completed.

○

Scenario 3: Check if a customer has sufficient balance before making a transaction.

- **Question:** Write a function **HasSufficientBalance** that takes an account ID and an amount as input and returns a boolean indicating whether the account has at least the specified amount.

```

SQL> CREATE OR REPLACE FUNCTION HasSufficientBalance(
  2     p_accountid IN Accounts.AccountID%TYPE,
  3     p_amount     IN Accounts.Balance%TYPE
  4 )
  5 RETURN BOOLEAN
  6 IS
  7     v_balance Accounts.Balance%TYPE;
  8 BEGIN
  9     SELECT Balance
 10     INTO v_balance
 11     FROM Accounts
 12     WHERE AccountID = p_accountid;
 13
 14     RETURN v_balance >= p_amount;
 15
 16 EXCEPTION
 17     WHEN NO_DATA_FOUND THEN
 18         RETURN FALSE;
 19 END;
20 /

Function created.

SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
  2     v_result BOOLEAN;
  3 BEGIN
  4     v_result := HasSufficientBalance(1,5000);
  5
  6     IF v_result THEN
  7         DBMS_OUTPUT.PUT_LINE('Sufficient Balance');
  8     ELSE
  9         DBMS_OUTPUT.PUT_LINE('Insufficient Balance');
 10     END IF;
 11 END;
 12 /

Insufficient Balance

PL/SQL procedure successfully completed.

```

○

Exercise 5: Triggers

Scenario 1: Automatically update the last modified date when a customer's record is updated.

- **Question:** Write a trigger **UpdateCustomerLastModified** that updates the LastModified column of the Customers table to the current date whenever a customer's record is updated.

```

SQL> CREATE OR REPLACE TRIGGER UpdateCustomerLastModified
  2 BEFORE UPDATE
  3 ON Customers
  4 FOR EACH ROW
  5 BEGIN
  6     :NEW.LastModified := SYSDATE;
  7 END;
  8 /

Trigger created.

```

○

○

Scenario 2: Maintain an audit log for all transactions.

- **Question:** Write a trigger **LogTransaction** that inserts a record into an AuditLog table whenever a transaction is inserted into the Transactions table.
-

```
SQL> CREATE TABLE AuditLog (  
2     LogID NUMBER PRIMARY KEY,  
3     TransactionID NUMBER,  
4     LogDate DATE,  
5     Action VARCHAR2(20)  
6 );  
  
Table created.  
  
SQL> CREATE OR REPLACE TRIGGER LogTransaction  
2 AFTER INSERT  
3 ON Transactions  
4 FOR EACH ROW  
5 BEGIN  
6     INSERT INTO AuditLog (  
7         TransactionID,  
8         LogDate,  
9         Action  
10    )  
11    VALUES (  
12        :NEW.TransactionID,  
13        SYSDATE,  
14        'INSERT'  
15    );  
16 END;  
17 /  
  
Trigger created.
```

Scenario 3: Enforce business rules on deposits and withdrawals.

- **Question:** Write a trigger **CheckTransactionRules** that ensures withdrawals do not exceed the balance and deposits are positive before inserting a record into the Transactions table.

```

SQL> CREATE OR REPLACE TRIGGER CheckTransactionRules
2  BEFORE INSERT
3  ON Transactions
4  FOR EACH ROW
5  DECLARE
6      v_balance Accounts.Balance%TYPE;
7  BEGIN
8      IF :NEW.TransactionType = 'DEPOSIT' THEN
9          IF :NEW.Amount <= 0 THEN
10             RAISE_APPLICATION_ERROR(
11                 -20001,
12                 'Deposit amount must be positive.'
13             );
14         END IF;
15     ELSIF :NEW.TransactionType = 'WITHDRAWAL' THEN
16         SELECT Balance
17         INTO v_balance
18         FROM Accounts
19         WHERE AccountID = :NEW.AccountID;
20
21         IF :NEW.Amount > v_balance THEN
22             RAISE_APPLICATION_ERROR(
23                 -20002,
24                 'Insufficient balance.'
25             );
26         END IF;
27     END IF;
28 END;
29 /

```

Trigger created.

○

Exercise 6: Cursors

Scenario 1: Generate monthly statements for all customers.

- **Question:** Write a PL/SQL block using an explicit cursor **GenerateMonthlyStatements** that retrieves all transactions for the current month and prints a statement for each customer.

```

SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
2     CURSOR GenerateMonthlyStatements IS
3         SELECT TransactionID,
4                AccountID,
5                Amount,
6                TransactionDate
7         FROM Transactions
8         WHERE EXTRACT(MONTH FROM TransactionDate) =
9               EXTRACT(MONTH FROM SYSDATE)
10        AND EXTRACT(YEAR FROM TransactionDate) =
11              EXTRACT(YEAR FROM SYSDATE);
12
13     v_rec GenerateMonthlyStatements%ROWTYPE;
14
15 BEGIN
16     OPEN GenerateMonthlyStatements;
17
18     LOOP
19         FETCH GenerateMonthlyStatements INTO v_rec;
20
21         EXIT WHEN GenerateMonthlyStatements%NOTFOUND;
22
23         DBMS_OUTPUT.PUT_LINE(
24             'Account ID: ' || v_rec.AccountID ||
25             ', Transaction ID: ' || v_rec.TransactionID ||
26             ', Amount: ' || v_rec.Amount ||
27             ', Date: ' || TO_CHAR(v_rec.TransactionDate, 'DD-MON-YYYY')
28         );
29     END LOOP;
30
31     CLOSE GenerateMonthlyStatements;
32 END;
33 /
Account ID: 1, Transaction ID: 1, Amount: 200, Date: 23-JUN-2026
Account ID: 2, Transaction ID: 2, Amount: 300, Date: 23-JUN-2026

PL/SQL procedure successfully completed.

```

○

Scenario 2: Apply annual fee to all accounts.

- **Question:** Write a PL/SQL block using an explicit cursor **ApplyAnnualFee** that deducts an annual maintenance fee from the balance of all accounts.

```

SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
2     CURSOR ApplyAnnualFee IS
3         SELECT AccountID, Balance
4         FROM Accounts
5         FOR UPDATE;
6
7     v_rec ApplyAnnualFee%ROWTYPE;
8     v_fee NUMBER := 100;
9
10 BEGIN
11     OPEN ApplyAnnualFee;
12
13     LOOP
14         FETCH ApplyAnnualFee INTO v_rec;
15
16         EXIT WHEN ApplyAnnualFee%NOTFOUND;
17
18         UPDATE Accounts
19         SET Balance = Balance - v_fee
20         WHERE CURRENT OF ApplyAnnualFee;
21
22     END LOOP;
23
24     CLOSE ApplyAnnualFee;
25
26     COMMIT;
27
28     DBMS_OUTPUT.PUT_LINE(
29         'Annual fee applied successfully.'
30     );
31 END;
32 /
Annual fee applied successfully.

PL/SQL procedure successfully completed.

```

○

Scenario 3: Update the interest rate for all loans based on a new policy.

- **Question:** Write a PL/SQL block using an explicit cursor **UpdateLoanInterestRates** that fetches all loans and updates their interest rates based on the new policy.

```

SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
  2     CURSOR UpdateLoanInterestRates IS
  3         SELECT LoanID, InterestRate
  4             FROM Loans
  5             FOR UPDATE;
  6
  7     v_rec UpdateLoanInterestRates%ROWTYPE;
  8
  9 BEGIN
10     OPEN UpdateLoanInterestRates;
11
12     LOOP
13         FETCH UpdateLoanInterestRates INTO v_rec;
14
15         EXIT WHEN UpdateLoanInterestRates%NOTFOUND;
16
17         UPDATE Loans
18             SET InterestRate = InterestRate + 0.5
19             WHERE CURRENT OF UpdateLoanInterestRates;
20
21     END LOOP;
22
23     CLOSE UpdateLoanInterestRates;
24
25     COMMIT;
26
27     DBMS_OUTPUT.PUT_LINE(
28         'Interest rates updated successfully.'
29     );
30 END;
31 /
Interest rates updated successfully.

PL/SQL procedure successfully completed.

SQL> select * from loans;

   LOANID CUSTOMERID LOANAMOUNT INTERESTRATE STARTDATE ENDDATE
-----
         1          1        5000          5.5 23-JUN-26 23-JUN-31

```

○

Exercise 7: Packages

Scenario 1: Group all customer-related procedures and functions into a package.

- **Question:** Create a package **CustomerManagement** with procedures for adding a new customer, updating customer details, and a function to get customer balance.

```
SQL> CREATE OR REPLACE PACKAGE CustomerManagement AS
2
3     PROCEDURE AddNewCustomer(
4         p_customerid IN Customers.CustomerID%TYPE,
5         p_name        IN Customers.Name%TYPE,
6         p_dob         IN Customers.DOB%TYPE,
7         p_balance     IN Customers.Balance%TYPE
8     );
9
10    PROCEDURE UpdateCustomer(
11        p_customerid IN Customers.CustomerID%TYPE,
12        p_name        IN Customers.Name%TYPE
13    );
14
15    FUNCTION GetCustomerBalance(
16        p_customerid IN Customers.CustomerID%TYPE
17    ) RETURN NUMBER;
18
19 END CustomerManagement;
20 /
```

Package created.

○

```

SQL> CREATE OR REPLACE PACKAGE BODY CustomerManagement AS
2
3     PROCEDURE AddNewCustomer(
4         p_customerid IN Customers.CustomerID%TYPE,
5         p_name        IN Customers.Name%TYPE,
6         p_dob         IN Customers.DOB%TYPE,
7         p_balance     IN Customers.Balance%TYPE
8     )
9     IS
10    BEGIN
11        INSERT INTO Customers(
12            CustomerID,
13            Name,
14            DOB,
15            Balance,
16            LastModified,
17            IsVIP
18        )
19        VALUES(
20            p_customerid,
21            p_name,
22            p_dob,
23            p_balance,
24            SYSDATE,
25            'FALSE'
26        );
27
28        COMMIT;
29    END AddNewCustomer;
30
31    PROCEDURE UpdateCustomerDetails(
32        p_customerid IN Customers.CustomerID%TYPE,
33        p_name        IN Customers.Name%TYPE,
34        p_balance     IN Customers.Balance%TYPE
35    )
36    IS
37    BEGIN
38        UPDATE Customers
39        SET Name = p_name,
40            Balance = p_balance,
41            LastModified = SYSDATE
42        WHERE CustomerID = p_customerid;
43
44        COMMIT;
45    END UpdateCustomerDetails;
46
47    FUNCTION GetCustomerBalance(
48        p_customerid IN Customers.CustomerID%TYPE
49    )
50    RETURN NUMBER
51    IS
52        v_balance Customers.Balance%TYPE;
53    BEGIN
54        SELECT Balance
55        INTO v_balance
56        FROM Customers
57        WHERE CustomerID = p_customerid;
58
59        RETURN v_balance;
60    END GetCustomerBalance;
61
62 END CustomerManagement;
63 /

```

○ **Scenario 2:** Create a package to manage employee data.

- **Question:** Write a package **EmployeeManagement** with procedures to hire new employees, update employee details, and a function to calculate annual salary.

```

SQL> CREATE OR REPLACE PACKAGE EmployeeManagement AS
2
3     PROCEDURE HireEmployee(
4         p_empid IN Employees.EmployeeID%TYPE,
5         p_name IN Employees.Name%TYPE,
6         p_salary IN Employees.Salary%TYPE
7     );
8
9     PROCEDURE UpdateEmployee(
10        p_empid IN Employees.EmployeeID%TYPE,
11        p_salary IN Employees.Salary%TYPE
12    );
13
14    FUNCTION CalculateAnnualSalary(
15        p_empid IN Employees.EmployeeID%TYPE
16    )
17    RETURN NUMBER;
18
19 END EmployeeManagement;
20 /

```

Package created.

```

SQL> CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS
2
3     PROCEDURE HireEmployee(
4         p_empid IN Employees.EmployeeID%TYPE,
5         p_name IN Employees.Name%TYPE,
6         p_salary IN Employees.Salary%TYPE
7     )
8     IS
9     BEGIN
10        INSERT INTO Employees(
11            EmployeeID,
12            Name,
13            Salary
14        )
15        VALUES(
16            p_empid,
17            p_name,
18            p_salary
19        );
20
21        COMMIT;
22    END HireEmployee;
23
24    PROCEDURE UpdateEmployee(
25        p_empid IN Employees.EmployeeID%TYPE,
26        p_salary IN Employees.Salary%TYPE
27    )
28    IS
29    BEGIN
30        UPDATE Employees
31        SET Salary = p_salary
32        WHERE EmployeeID = p_empid;
33
34        COMMIT;
35    END UpdateEmployee;
36
37    FUNCTION CalculateAnnualSalary(
38        p_empid IN Employees.EmployeeID%TYPE
39    )
40    RETURN NUMBER
41    IS
42        v_salary Employees.Salary%TYPE;
43    BEGIN
44        SELECT Salary
45        INTO v_salary
46        FROM Employees
47        WHERE EmployeeID = p_empid;
48
49        RETURN v_salary * 12;
50    END CalculateAnnualSalary;
51
52 END EmployeeManagement;
53 /

```

Package body created.

Scenario 3: Group all account-related operations into a package.

- **Question:** Create a package **AccountOperations** with procedures for opening a new account, closing an account, and a function to get the total balance of a customer across all accounts.

```

SQL> CREATE OR REPLACE PACKAGE AccountOperations AS
2
3     PROCEDURE OpenAccount(
4         p_accountid IN Accounts.AccountID%TYPE,
5         p_customerid IN Accounts.CustomerID%TYPE,
6         p_type IN Accounts.AccountType%TYPE,
7         p_balance IN Accounts.Balance%TYPE
8     );
9
10    PROCEDURE CloseAccount(
11        p_accountid IN Accounts.AccountID%TYPE
12    );
13
14    FUNCTION GetTotalBalance(
15        p_customerid IN Accounts.CustomerID%TYPE
16    )
17    RETURN NUMBER;
18
19 END AccountOperations;
20 /

```

Package created.

```

SQL> CREATE OR REPLACE PACKAGE BODY AccountOperations AS
2
3     PROCEDURE OpenAccount(
4         p_accountid IN Accounts.AccountID%TYPE,
5         p_customerid IN Accounts.CustomerID%TYPE,
6         p_type IN Accounts.AccountType%TYPE,
7         p_balance IN Accounts.Balance%TYPE
8     )
9     IS
10    BEGIN
11        INSERT INTO Accounts(
12            AccountID,
13            CustomerID,
14            AccountType,
15            Balance,
16            LastModified
17        )
18        VALUES(
19            p_accountid,
20            p_customerid,
21            p_type,
22            p_balance,
23            SYSDATE
24        );
25
26        COMMIT;
27    END OpenAccount;
28
29    PROCEDURE CloseAccount(
30        p_accountid IN Accounts.AccountID%TYPE
31    )
32    IS
33    BEGIN
34        DELETE FROM Accounts
35        WHERE AccountID = p_accountid;
36
37        COMMIT;
38    END CloseAccount;
39
40    FUNCTION GetTotalBalance(
41        p_customerid IN Accounts.CustomerID%TYPE
42    )
43    RETURN NUMBER
44    IS
45        v_total NUMBER;
46    BEGIN
47        SELECT NVL(SUM(Balance),0)
48        INTO v_total
49        FROM Accounts
50        WHERE CustomerID = p_customerid;
51
52        RETURN v_total;
53    END GetTotalBalance;
54
55 END AccountOperations;
56 /

```

Package body created.

Schema to be Created

```
CREATE TABLE Customers (  
    CustomerID NUMBER PRIMARY KEY,  
    Name VARCHAR2(100),  
    DOB DATE,  
    Balance NUMBER,  
    LastModified DATE  
);
```

```
CREATE TABLE Accounts (  
    AccountID NUMBER PRIMARY KEY,  
    CustomerID NUMBER,  
    AccountType VARCHAR2(20),  
    Balance NUMBER,  
    LastModified DATE,  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
);
```

```
CREATE TABLE Transactions (  
    TransactionID NUMBER PRIMARY KEY,  
    AccountID NUMBER,  
    TransactionDate DATE,  
    Amount NUMBER,  
    TransactionType VARCHAR2(10),  
    FOREIGN KEY (AccountID) REFERENCES Accounts(AccountID)  
);
```

```
CREATE TABLE Loans (  
    LoanID NUMBER PRIMARY KEY,  
    CustomerID NUMBER,  
    LoanAmount NUMBER,  
    InterestRate NUMBER,  
    StartDate DATE,  
    EndDate DATE,  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
);
```

```
CREATE TABLE Employees (  
    EmployeeID NUMBER PRIMARY KEY,  
    Name VARCHAR2(100),  
    Position VARCHAR2(50),  
    Salary NUMBER,  
    Department VARCHAR2(50),  
    HireDate DATE  
);
```

Example Scripts for Sample Data Insertion

```
INSERT INTO Customers (CustomerID, Name, DOB, Balance, LastModified)
VALUES (1, 'John Doe', TO_DATE('1985-05-15', 'YYYY-MM-DD'), 1000, SYSDATE);
```

```
INSERT INTO Customers (CustomerID, Name, DOB, Balance, LastModified)
VALUES (2, 'Jane Smith', TO_DATE('1990-07-20', 'YYYY-MM-DD'), 1500, SYSDATE);
```

```
INSERT INTO Accounts (AccountID, CustomerID, AccountType, Balance, LastModified)
VALUES (1, 1, 'Savings', 1000, SYSDATE);
```

```
INSERT INTO Accounts (AccountID, CustomerID, AccountType, Balance, LastModified)
VALUES (2, 2, 'Checking', 1500, SYSDATE);
```

```
INSERT INTO Transactions (TransactionID, AccountID, TransactionDate, Amount, TransactionType)
VALUES (1, 1, SYSDATE, 200, 'Deposit');
```

```
INSERT INTO Transactions (TransactionID, AccountID, TransactionDate, Amount, TransactionType)
VALUES (2, 2, SYSDATE, 300, 'Withdrawal');
```

```
INSERT INTO Loans (LoanID, CustomerID, LoanAmount, InterestRate, StartDate, EndDate)
VALUES (1, 1, 5000, 5, SYSDATE, ADD_MONTHS(SYSDATE, 60));
```

```
INSERT INTO Employees (EmployeeID, Name, Position, Salary, Department, HireDate)
VALUES (1, 'Alice Johnson', 'Manager', 70000, 'HR', TO_DATE('2015-06-15', 'YYYY-MM-DD'));
```

```
INSERT INTO Employees (EmployeeID, Name, Position, Salary, Department, HireDate)
VALUES (2, 'Bob Brown', 'Developer', 60000, 'IT', TO_DATE('2017-03-20', 'YYYY-MM-DD'));
```